

10 单机与并行机：一条规则为什么就是最优

第 9 章里反复出现 SPT、WSPT、EDD、Moore–Hodgson，它们是“一条规则就得到最优”的例子。这一章逐个证明它们为什么对，用的是同一个工具：交换论证。再看机器多了以后，快速的列表调度离最优最多差多少，以及允许中断能带来什么。

学完本章，你能

- 1. 用交换论证证明 SPT、WSPT、EDD 分别是 $\sum C_j$ 、 $\sum w_j C_j$ 、 $L_{\max}$ 的最优规则，并能预言“规则与目标不匹配”时交换会让目标变差。
- 2. 手写 Moore–Hodgson，解释为什么“超期就移走最长的”是对的，以及它为什么不能直接推广到带权重的情形。
- 3. 说明并行机上列表调度的近似比 $2 - 1/m$ 和 LPT 的 $4/3 - 1/(3m)$ ，构造让它们取等号的最坏实例。
- 4. 用 McNaughton 绕圈法解可中断的并行机问题，用子集和动态规划解 $P2 \parallel C_{\max}$ ，并理解“弱 NP 难”的意思。

本章信息

用时	阅读与交互 70–100 分钟，Lab 2.5–3 小时
前置	第 9 章（三段式记号、目标函数、复杂度地图）。会读 Python 代码。
对拍	SciPy 1.17.1 · HiGHS 1.12.0；本章的算法由页面内置代码实现，不依赖求解器。
配套	Lab 代码包：5 个练习 + 1 个挑战，含三条规则、Moore–Hodgson、相邻交换、并行机最坏实例与绕圈法，pytest 自动判分

PART 1 数学规划基础	PART 2 组合优化	PART 3 调度理论	PART 4 路径与波次
PART 5 大规模方法	PART 6 启发式	PART 7 多智能体时空规划	PART 8 动态与不确定

第三部分 调度理论：调度问题的语言 → 单机与并行机（本章） → 流水车间与作业车间 → 资源与时间窗约束。

10.1 交换论证与三条规则

单机上排 n 个作业有 $n!$ 种顺序，却常常只需要按某个量排一下序。为什么这样是对的？答案是一个很朴素的想法：如果一个顺序里有相邻的两个作业“排反了”，把它们换过来，目标不会变差。一直换下去，就走到规则给出的顺序。

取顺序里相邻的两个作业 a （在前）和 b （在后），它们开始于时刻 t 。交换 a 和 b 只改变这两个作业的完工时间，别的作业的完工时间都不受影响（前面的没动，后面的作业开始时刻仍是 $t + p_a + p_b$ ）。

SPT : 1 || $\sum C_j$

交换前两个作业的完工时间是 $t + p_a$ 和 $t + p_a + p_b$ ；交换后是 $t + p_b$ 和 $t + p_a + p_b$ 。 $\sum C$ 的变化 $= p_b - p_a$ 。

若 $p_a > p_b$ ，交换让 $\sum C$ 减少。所以最优顺序里不会有“长的在短的前面”的相邻对，即加工时间从小到大。

WSPT (Smith 规则) : 1 || $\sum w_j C_j$

同样的算法，变化 $= w_a p_b - w_b p_a$ 。

它 ≤ 0 当且仅当 $p_b/w_b \leq p_a/w_a$ 。所以按 p_j/w_j 从小到大排（“单位权重的加工时间”越小越先做）。权重全为 1 就是 SPT。

EDD (Jackson 规则) : 1 || $L_{\max}$

设 $d_a > d_b$ ， a 排在前面。交换前， b 的延迟是 $L_b = t + p_a + p_b - d_b$ 。交换后 b 的延迟更小， a 的延迟 $t + p_a + p_b - d_a$ 也小于交换前 b 的延迟（因为 $d_a > d_b$ ）。

这两个作业的最大延迟不会变大，所以最大延迟不会变大。最优顺序里可以让交货期早的先做。

结论：从任意一个顺序出发，只要还有“排反了”的相邻对，就交换，目标不变差；交换总会结束（每次交换减少逆序对），结束的顺序就是规则给出的顺序。所以规则给出的顺序不比任何一个起点差，也就是最优。这就是**交换论证**（interchange argument）。证明不需要枚举，只需要看两个相邻的作业。

先预测 单机上，相邻的两个作业：加工时间 7 的 **a** 排在加工时间 2 的 **b** 前面。交换它们（**b** 排到 **a** 前面）， ΣC_j 会怎样？前面和后面的作业不受影响。

减少 5

减少 9

不变

增加 5

预测正确。 减少 5。交换前完工时间是 $t+7$ 和 $t+9$ （和 $2t+16$ ），交换后是 $t+2$ 和 $t+9$ （和 $2t+11$ ）。一般地，变化 = $p_b - p_a$ 。所以长的排在短的前面总是可以改进，这就是 SPT。下面的图已经解锁。

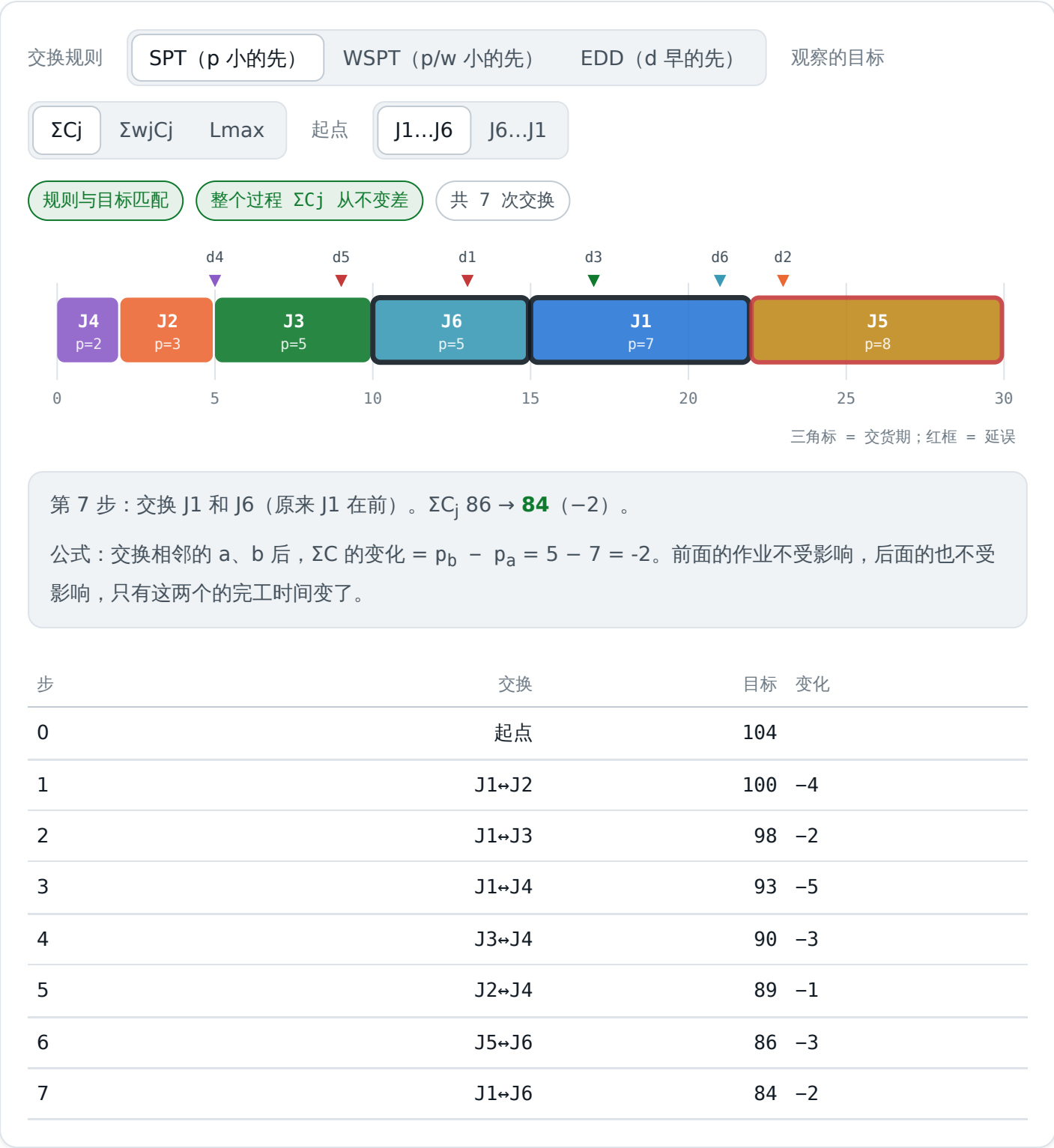


图 10-1 交换论证的动画（第 9 章的 6 个托盘）。每一步：从左往右找第一个“后一个按规则应当排在前面”的相邻对，交换。黑框是这一步交换的两个作业。规则与目标匹配时（SPT-ΣC、WSPT-Σw_jC_j、EDD-L_{max}）目标从不变差；换成不匹配的组合，交换有时会让目标变差。点表里的一行可以跳到那一步。

读图

SPT 观察 ΣC，从 J1...J6 出发：7 次交换，目标 104、100、98、93、90、89、86、84，每次都下降，最后是最优的 84。每一步的降幅正好是公式里的 p_a - p_b。

切到 **WSPT** 观察 $\sum w_j C_j$: 8 次交换, 322 一路降到 234, 从不变差。切到 **EDD** 观察 $L_{\max}$: 目标不变差, $16 \rightarrow 7$, 中间有不少步保持 16 不变, 因为最大延迟由别的作业决定。

再试不匹配的组合。把规则选成 SPT, 目标选成 $\sum w_j C_j$: 交换按“p 小的先”, 但目标关心权重。从 J1...J6 出发, 第 6 步 252 变成 256, 变差了, 并且最后停在 252, 比最优的 234 高。把规则选成 EDD, 目标选成 $\sum C_j$, 第 1 步就变差, 最后是 108, 比 SPT 的 84 差得多。规则只对它对应的目标有保证, 这也是第 9 章说“选目标就是选规则”的原因。

10.2 Moore-Hodgson : 让延误的作业最少

前三条规则都是“排个序”。 $\sum U_j$ (延误作业个数) 不一样: 它不在乎晚多少, 只在乎晚不晚。既然有的作业注定晚, 不如干脆让它们晚到底, 把机器的时间让给还有希望的作业。

Moore-Hodgson (1968) 的做法:

- 1 按交货期从早到晚 (EDD) 依次把作业加进“准时集合”。
- 2 每加一个, 检查现在的完工时间是否超过这个作业的交货期。
- 3 超了, 就从准时集合里移走加工时间最长的一个 (可以是刚加进来的)。被移走的作业放弃, 排在最后, 接受延误。

为什么移最长的? 要让剩下的作业尽早完成, 就要腾出最多的时间。移走任何一个作业都能让后面的完工时间提早, 移走最长的提早得最多, 而且被放弃的代价 (少一个准时作业) 对每个作业都一样, 都是 1。这就是“个数”目标的特点: 所有作业的价值都是 1, 所以要放弃“占地方最大”的那个。

先预测 第 9 章的 6 个托盘 ($p = 7, 3, 5, 2, 8, 5$; $d = 13, 23, 17, 5, 9, 21$), 按 **EDD** 顺序直接排, 延误的作业有 5 个。用 **Moore-Hodgson**, 最少能压到几个?

4 个

3 个

2 个

1 个

预测正确。 1 个。EDD 顺序里 J5 ($p = 8$, 交货期 9) 拖累了后面所有作业。Moore-Hodgson 发现超期后, 把已选作业里最长的 J5 移走, 其他 5 个全部按期。下面的图已经解锁。

例子

例 1：本章的 6 个托盘

例 2：7 个作业，交货期更紧

按期作业

5

J4 J1 J3 J6 J2

已移走（延误）

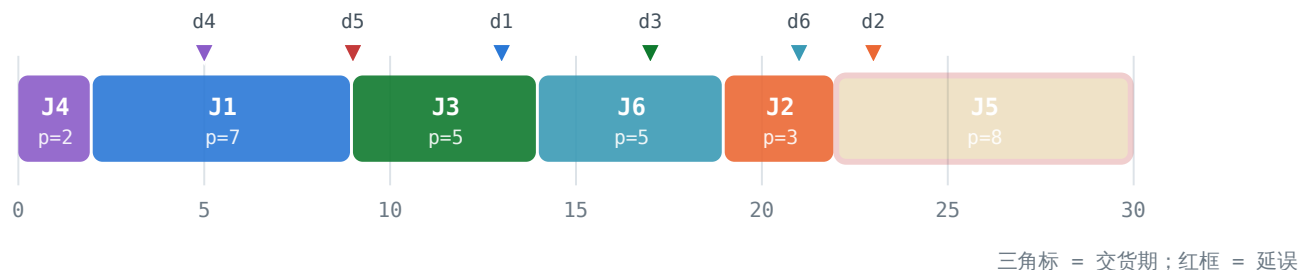
1

J5

EDD 顺序直接排的延误数

5

对照：Moore-Hodgson 最优 1



结束。 延误作业 J5，共 **1** 个；按期作业 J4 → J1 → J3 → J6 → J2 排在前面，延误的放到最后。对照 EDD 顺序直接排：延误 5 个。

图 10-2 Moore-Hodgson 的过程。作业按 EDD 顺序逐个加入，图里是当前准时集合的排法（黑框是刚加入的作业）；一旦超期，就移走集合里最长的作业。最后一步灰色的是被放弃的（延误）作业。

读图

例 1：EDD 顺序是 J4、J5、J1、J3、J6、J2。加入 J4 (2)，加入 J5，完工时间 10，超过它的交货期 9，超期：在 {J4, J5} 里移走最长的 J5 (8)，完工时间回到 2。之后 J1、J3、J6、J2 都按期。只延误 J5 一个，是最优的。直接按 EDD 排，延误 5 个。

例 2 更紧：7 个作业，EDD 直接排延误 6 个；Moore-Hodgson 依次移走 J3、J2、J7，延误 3 个，仍是最优（枚举核对）。

别推广过头

加上权重， $1 \parallel \sum w_j U_j$ （最小化延误作业的权重之和）就是背包问题的推广，是弱 NP 难的（第 9 章的地图里是橙色）。“移走最长的”不再对：有时该移走一个短的但权重小的。同样， $1 \parallel \sum T_j$ （延误时间之和）也是 NP 难的。看起来很像的目标，难度可以差得很远。

10.3 并行机：列表调度有多差

机器不止一台，最小化 $C_{\max}$ （让最后一个作业尽快完成）就成了均衡负荷的问题。P2 || $C_{\max}$ 已经是 NP 难，没有简单的规则保证最优。退而求其次：一个很快的算法，最坏能比最优差多少？这个“最多差多少倍”叫近似比。

列表调度（第 9 章）：按给定的顺序，把每个作业放到当前完工最早的机器上。它的近似比有一个简洁的证明（Graham, 1966）：设最后完工的作业是 l 。它开始的时候，所有机器都是忙的（否则会更早放上去），所以此时每台机器的负荷至少是它的开始时刻 S_l ，于是 $S_l \leq (\text{总量} - p_l)/m$ 。因此

$$\text{Graham 界 } C_{\max} = S_l + p_l \leq \text{总量}/m + (1 - 1/m)p_l \leq (2 - 1/m) \cdot \text{OPT} \quad \begin{array}{l} \text{用到 } \text{OPT} \geq \text{总量}/m \\ \text{和 } \text{OPT} \geq p_l \end{array}$$

如果先把作业按加工时间从大到小排（**LPT**, longest processing time first），再做列表调度，界会好很多： $C_{\max} \leq (4/3 - 1/(3m)) \cdot \text{OPT}$ （Graham, 1969）。这两个界都是紧的：能构造出让它们取等号的实例。

先预测 4 台相同的机器。先来 12 个长度为 1 的作业，最后来一个长度为 4 的作业，按这个顺序做列表调度（每个作业放到完工最早的机器）。 $C_{\max}$ 是多少？（最优是 4）

4 5 6 7

预测正确。 7。前 12 个单位作业把 4 台机器各铺到 3，长作业接在任何一台后面， $3 + 4 = 7$ ；最优 4，比值 $1.75 = 2 - 1/4$ ，正好是 Graham 界。下面的图已经解锁。

实例

列表调度的最坏实例

LPT 的最坏实例

可中断：绕圈法

机器数 $m = 4$ 列表调度的 $C_{\max}$ **7**

先排 12 个单位作业，最后来长度 4 的

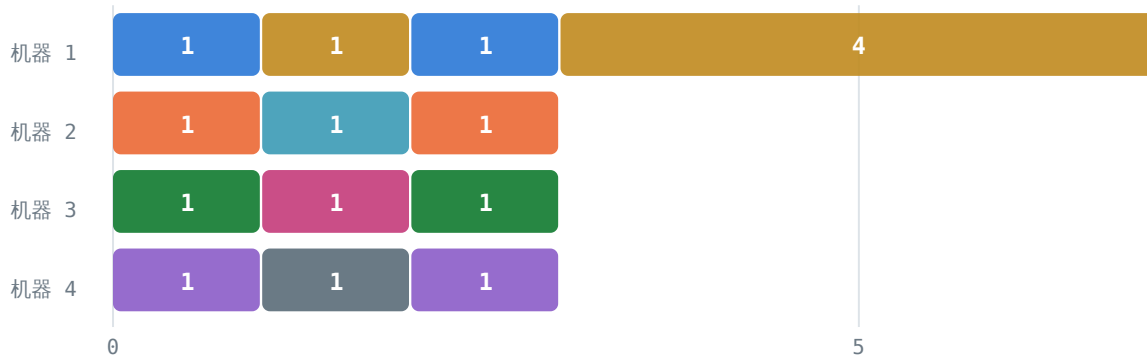
最优 $C_{\max}$ **4**

下面这张图给出构造出来的最优排法

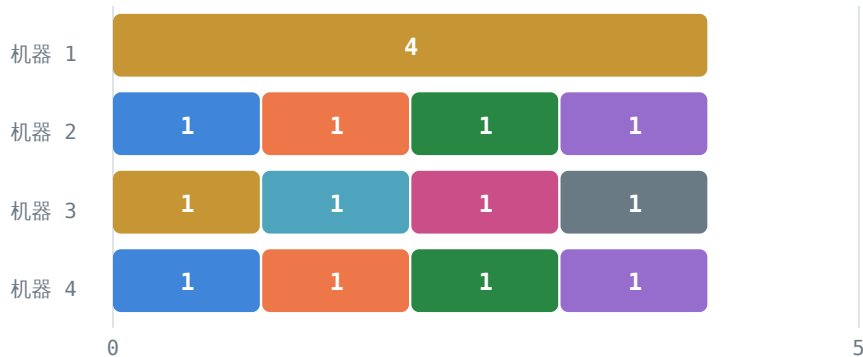
比值

1.75

理论上界 $2 - 1/m = 1.75$



对照：最优排法



前面的 12 个单位作业把每台机器都平均地铺到 3，最后来的长作业（长度 4）落在任意一台上，完工时间 $3 + 4 = 7$ 。最优是让长作业独占一台，其余 12 个单位作业平分到剩下 3 台机器，每台 4， $C_{\max} = 4$ 。比值 $(2m - 1)/m = 2 - 1/m$ ，刚好等于 Graham 的上界。这个界是紧的。

图 10-3 相同并行机。方块里是作业的加工时间。“列表调度的最坏实例”：先来 $m(m-1)$ 个单位作业，最后来一个长度 m 的作业。“LPT 的最坏实例”： $2m+1$ 个作业。第三个模式是下一节的绕圈法。最优排法是构造出来的（并在 $m \leq 3$ 时用枚举核对过）。

读图

列表调度, $m = 4$: 12 个单位作业把 4 台机器各铺到 3, 最后的长作业 (长度 4) 接在任何一台后面, $C_{\max} = 3 + 4 = 7$ 。最优是让长作业独占一台, 12 个单位作业平分给另外 3 台, 每台 4, $C_{\max} = 4$ 。比值 $7/4 = 1.75 = 2 - 1/4$, 正好是 Graham 界。拖动 m : 比值 $(2m-1)/m$ 随机器数增加, 趋向 2。

LPT, $m = 4$: 9 个作业, LPT 得到 15, 最优 12, 比值 $1.25 = 4/3 - 1/12$, 正好是 LPT 的界。所以 LPT 已经很好, 最多差三分之一; 列表调度不排序, 最坏差近一倍。这个对比说明**排序这一步有多值钱**。

10.4 可中断与弱 NP 难

前面的所有作业都不能中断。允许中断 (prmp) 之后, 同一个作业可以先做一部分, 换到别的机器 (或过一会儿) 再做完, 问题就大不一样。

McNaughton 绕圈法 (1959) : 允许中断时, $P \mid \text{prmp} \mid C_{\max}$ 的最优值恰好是下界

可中断的下界 $C_{\max} = \max(\max_j p_j, (\sum p_j)/m)$ 任何排法都不可能更小; 绕圈法能达到

做法: 把作业排成一行, 第 1 台机器装到 C , 再换第 2 台, 一个作业装不下就断开, 剩下的部分从下一台机器的开头继续。同一个作业的两段一段在前一台机器的末尾, 一段在后一台的开头, 时间不重叠 (因为 $C \geq$ 它的长度)。图 10-3 切到“可中断: 绕圈法”看一个例子: 3 台机器, 作业 9、9、9、3, 总量 30, $C_{\max} = 10$; 不能中断时最优是 12。

这就是复杂度地图里 $P \mid \text{prmp} \mid C_{\max}$ 是绿色、 $P \parallel C_{\max}$ 是橙色或红色的原因: **“不能中断”是难点所在**。物流里“一个托盘可以拆成两批搬”“一辆车的任务可以换人接着做”对应可中断; 托盘不能拆、车不能中途换, 就是不可中断。

$P2 \parallel C_{\max}$: 弱 NP 难

不可中断时, 两台机器上的 $C_{\max}$ 是子集和问题: 把作业分成两堆, 让较大的一堆尽量小, 也就是找一堆作业的加工时间之和 s 尽量接近总量的一半 (不超过)。用一维的动态规划: $\text{reach}[s]$ 表示能否凑出 s , 逐个作业更新, 时间 $O(n \cdot \text{总量})$ 。最优 $C_{\max} = \text{总量} - \text{最大的可达 } s$ 。

它是**伪多项式**的：时间与数值的大小（总量）成正比，而不是与它的位数成正比。加工时间是秒、总量几千时，没有问题；数值是毫秒、总量上亿时就慢了。这就是“弱 NP 难”的含义。对比： m 是变量的 $P || C_{\max}$ 是**强 NP 难**的（3-划分），连伪多项式算法也没有（除非 $P = NP$ ）。Lab 挑战让你实现这个动态规划，并在 300 个作业上运行。

10.5 在项目里的位置

这一章的规则都很简单，价值在于它们是有**证明的基准**：你的调度系统在简化的情形下，至少应当不比它们差。

你在乎的	规则	保证
一台提升机，托盘平均等待最短	SPT（有权重用 WSPT）	最优（没有释放时间）
每个托盘有发运截止时间，最坏的一单别迟太多	EDD	$L_{\max}$ 最优
尽量多的托盘赶上截止时间	Moore–Hodgson	准时的个数最多
多台提升机、多辆车，负荷要均衡	LPT + 列表调度	$C_{\max}$ 至多是最优的 $4/3 - 1/(3m)$ 倍
可以把一个大任务拆成几批交给不同的车	McNaughton 绕圈法	$C_{\max}$ 最优（可中断）

两点提醒。第一，这些保证都假设作业**一开始就全部在那里**（没有释放时间）。任务陆续到达时，第 9 章的地图里 $1 || r_j || L_{\max}$ 、 $1 || r_j || \sum C_j$ 都是强 NP 难，规则只能当作在线策略（每次机器空闲时，从队列里按规则挑一个），没有最优保证；可中断的版本（SRPT、可中断 EDD）反而是多项式的，见第 30 章。第二，交换论证不只是证明规则用的：**它还告诉你 MIP 里可以加哪些有效不等式**。如果两个相邻作业按规则排反了不会更好，就可以在模型里禁止这种排法，减小搜索空间（第 11 章）。

10.6 小结与 Lab

- 1
- 交换论证：取相邻的两个作业交换，只有这两个的完工时间变化。若交换对目标不会变差，反复交换就走到规则给出的顺序。 $\sum C$ 的变化是 $p_b - p_a$ （SPT）， $\sum wC$ 的变化是 $w_a p_b - w_b p_a$ （WSPT），EDD 让最大延迟不变大。

- 2 规则只对它对应的目标有保证：用 SPT 的交换去优化带权重的 ΣwC ，或者用 EDD 的交换去优化 ΣC ，目标会变差。
- 3 Moore-Hodgson：按 EDD 加入，超期就移走已选作业里最长的，得到最少的延误作业个数。加上权重 (ΣwU) 或者改成延误时间之和 (ΣT) 就是 NP 难了。
- 4 并行机上，列表调度的近似比是 $2 - 1/m$ (证明：最后一个作业开始时所有机器都忙)，LPT 是 $4/3 - 1/(3m)$ ；两个界都是紧的，本章给出了取等号的实例。排序这一步把最坏情形从差近一倍降到差三分之一。
- 5 允许中断时， $P \mid \text{prmp} \mid C_{\max}$ 有最优的绕圈法， $C_{\max} = \max(\text{最长作业}, \text{总量}/m)$ ，多项式可解；不可中断的 $P \parallel C_{\max}$ 是 NP 难的。
- 6 $P2 \parallel C_{\max}$ 是子集和，有 $O(n \cdot \text{总量})$ 的伪多项式动态规划 (弱 NP 难)； m 是变量的 $P \parallel C_{\max}$ 是强 NP 难。
- 7 在项目里，这些规则是有证明的基准，也是在线策略的原型；它们的保证都建立在“作业一开始就全部就位”上，任务陆续到达时要另外分析。

术语表

中文	English	在本章的含义
交换论证	interchange (exchange) argument	交换相邻两个作业，证明目标不变差，从而规则给出最优
SPT、WSPT、EDD	shortest processing time, weighted SPT, earliest due date	p 小的先；p/w 小的先；d 早的先
Moore–Hodgson 算法	Moore–Hodgson algorithm	按 EDD 加入，超期就移走最长的作业
近似比	approximation ratio	算法的结果与最优值的最大比值
列表调度、LPT	list scheduling, longest processing time first	按顺序放到完工最早的机器；先按加工时间从大到小排序
紧的界	tight bound	存在让不等式取等号的实例
可中断	preemption (prmp)	一个作业可以分几段做，可以换机器
McNaughton 绕圈法	McNaughton's wrap-around rule	可中断时达到 $C_{\max}$ 下界的排法
伪多项式、弱/强 NP 难	pseudo-polynomial, weakly / strongly NP-hard	时间与数值成正比；有 / 没有伪多项式算法

对照教材

- Pinedo , *Scheduling: Theory, Algorithms, and Systems* (第 5 版 , Springer 2016) : 第 3 章单机 (SPT、WSPT、EDD、Moore–Hodgson , 交换论证)、第 5 章并行机 (LPT、McNaughton) 。
- Smith , “Various optimizers for single-stage production” (1956) : WSPT。Jackson , “Scheduling a production line to minimize maximum tardiness” (1955) : EDD。

Moore , “An n job, one machine sequencing algorithm for minimizing the number of late jobs” (Management Science , 1968) 。

- Graham , “Bounds for certain multiprocessing anomalies” (1966) 与 “Bounds on multiprocessing timing anomalies” (1969) : 列表调度与 LPT 的界。McNaughton , “Scheduling with deadlines and loss functions” (1959) 。
- Brucker , *Scheduling Algorithms* (第 5 版 , Springer 2007) : 单机与并行机的完整算法与复杂度。Williamson & Shmoys , *The Design of Approximation Algorithms* (2011) : 列表调度和 LPT 的近似分析 , 以及多项式近似方案。

本章 Lab

代码包 `ch10-single-parallel-lab.zip` 随本章一起发放。依赖是 `numpy`、`pytest` ; 判分用暴力枚举 , 不依赖你的代码。

练习	内容	对应
ex1	三条规则 : SPT、WSPT、EDD , 并列取下标小的 ; 先填 4 个预测	10.1
ex2	Moore-Hodgson : 最少的延误作业数	10.2
ex3	交换论证 : 相邻交换的变化量 ; 冒泡式交换到规则顺序 , 验证每一步不变差	10.1
ex4	并行机 : LPT、列表调度、两个最坏实例 ($2m-1$ 与 $4m-1$) 、McNaughton 绕圈法	10.3、10.4
ex5	审 5 个 AI 写的单机排序规则 : 不看源码 , 只靠运行结果归类问题	10.1、10.2
挑战	$P2 C_{\max}$ 的子集和动态规划 (300 个作业也很快)	10.4

```
pip install -r requirements.txt
pytest -q # 全部练习
pytest tests/test_ex4.py -q # 单个练习

# 确认判分本身没问题 : 参考答案应当全部通过
LAB_TARGET=solutions pytest -q
```

练习 1 要求先在 `PREDICTION` 里写下预测再运行 ; 跑完测试后 , 终端最后会把你的预测和实际结果并排列出来。

下一章

CHAPTER 11

流水车间与作业车间

一台机器或一组相同的机器，规则就够用了。当作业要依次经过好几台机器（入库 → 提升机 → 出库），问题一下子难了：两台机器串联的流水车间还有 Johnson 规则，三台以上就是强 NP 难。下一章用析取图表示作业车间的调度，再对比 MIP 和约束规划（CP-SAT）两种建模方式。

10.7 自测

8 道题，覆盖本章的全部要点。每题选完立即给出解析。

Q1 交换论证里，交换相邻的两个作业 **a**、**b**（**a** 原来在前），为什么只有这两个作业的完工时间会变？

前面的作业没有动；这两个作业合起来占用的时间不变，后面的作业开始时刻仍是 $t + p_a + p_b$

因为机器可以并行

因为完工时间与顺序无关

因为其他作业的加工时间为 0

正确。 交换只改变两个作业的先后，它们合起来仍占用 $p_a + p_b$ 的时间，所以后面的作业不受影响。这让证明只需要比较两个作业。

Q2 **WSPT** 规则按什么排序？为什么？

按 w 从大到小

按 p/w 从小到大：相邻交换的变化 $= w_a \cdot p_b - w_b \cdot p_a \leq 0$ 当且仅当 $p_b/w_b \leq p_a/w_a$

按 p 从小到大

按 d 从小到大

正确。 “单位权重的加工时间”小的先做。权重全为 1 时退化成 SPT。

Q3 用 SPT 的交换规则去优化 $\sum w_j C_j$ ，会怎样？

目标不变

仍然最优

每次交换都让目标变好

交换有时会让目标变差，最终也不一定最优

正确。 交换按 p 小的先，但 $\sum w_j C_j$ 关心的是 p/w 。图 10-1 里从 $J1 \dots J6$ 出发，第 6 步 252 变成 256，最后停在 252，比最优 234 高。规则只对它对应的目标有保证。

Q4 Moore-Hodgson 中，超期后为什么移走已选作业里加工时间最长的？

因为它的权重最大

因为这样最快

因为它的交货期最早

每个作业放弃的代价都是 1（少一个准时作业），要让后面的作业尽早完成，腾出的时间越多越好，最长的最多

正确。 $\sum U$ 只数个数，所有作业价值相同，所以放弃占地方最大的。有权重时价值不同，这个论证就不成立， $1 \parallel \sum wU$ 是弱 NP 难。

Q5 列表调度的近似比 $2 - 1/m$ 的证明用到了哪两件事？

最后一个作业开始时所有机器都忙（所以开始时刻 $\leq (\text{总量} - p_l)/m$ ），以及 $\text{OPT} \geq \text{总量}/m$ 和 $\text{OPT} \geq p_l$

作业都是单位长度

作业按 LPT 排序

机器数是偶数、作业数是奇数

正确。 $C_{\max} = S_l + p_l \leq (\text{总量} - p_l)/m + p_l = \text{总量}/m + (1 - 1/m)p_l \leq \text{OPT} + (1 - 1/m)\text{OPT}$ 。

Q6 图 10-3 里， $m = 4$ 时列表调度最坏实例的 $C_{\max}$ 是 7，最优是 4。这说明什么？

列表调度总是比最优差一倍

Graham 界 $2 - 1/m = 1.75$ 是紧的：7/4 正好等于它

$m = 4$ 时 LPT 也是 7

最优不可能是 4

正确。 取等号的实例说明这个界不能再改进。LPT 把作业先排序，同样的机器数下最坏情形是 $4/3 - 1/(3m) = 1.25$ 。

Q7 为什么允许中断以后， $P \mid \text{prmp} \mid C_{\max}$ 是多项式可解的，而 $P \parallel C_{\max}$ 是 NP 难的？

两者都是多项式的

中断让机器变快了

可中断时，绕圈法总能达到下界 $\max(\text{最长作业}, \text{总量}/m)$ ；不能中断时，作业整块放置，需要解装箱式的组合问题

因为可中断时作业更多

正确。 绕圈法把作业断开，每台机器正好装满 C 。不能中断就不能这样凑，需要选择哪些作业放在一起，这就是子集和 / 划分问题。

Q8 “ $P2 \parallel C_{\max}$ 是弱 NP 难， $P \parallel C_{\max}$ 是强 NP 难”，区别是？

两者没有区别

强 NP 难可以在多项式时间内解

弱的更难

弱 NP 难有伪多项式算法（时间与数值大小成正比，如子集和的动态规划）；强 NP 难连这样的算法也没有（除非 $P = NP$ ）

正确。 $P2 \parallel C_{\max}$ 用 $O(n \cdot \text{总量})$ 的动态规划解，数值不大时很快。 m 作为输入的 $P \parallel C_{\max}$ 由 3-划分推出，是强 NP 难的。

8 / 8 本章掌握得不错，可以去做 Lab 了。

数据说明：本章的全部数字由页面内置的相邻交换、Moore-Hodgson、列表调度、LPT、McNaughton 绕圈法和枚举实时计算，并与暴力枚举逐项对拍（环境：SciPy 1.17.1 · HiGHS 1.12.0）：随机实例上三条规则与 Moore-Hodgson 取得枚举最优，Graham 界与 LPT 界成立，最坏实例的比值为 $(2m-1)/m$ 与 $(4m-1)/(3m)$ ，都与 Lab 的判分数据同源。